Learning Burn with Rust

A coding tutorial with tensor visualisations

```
                    (
                  sequence(
                      [ ],
                      box(
                fill: rgb("#dceefb"),
              stroke: 0.8pt + rgb("#20639b"),
                     radius: 0% + 4pt,
                     inset: 0% + 7pt,
                      body: sequence(
                             [ ],
align(alignment: center, body: strong(body: [Rust data])),
                        linebreak(),
                             [ ],
          styled(child: [[rows, features]], ..),
                             [ ],
                             ),
                           ),
                          [ ],
                        sequence(
                      h(amount: 7pt),
                             [ ],
  styled(child: equation(block: false, body: [→]), ..),
                             [ ],
                      h(amount: 7pt),
                           ),
                          [ ],
                          ),
                        sequence(
                            [ ],
                            box(
                fill: rgb("#dceefb"),
              stroke: 0.8pt + rgb("#20639b"),
                     radius: 0% + 4pt,
                     inset: 0% + 7pt,
                      body: sequence(
                             [ ],
                           align(
                     alignment: center,
              body: strong(body: [Burn tensor]),
                             ),
                        linebreak(),
                             [ ],
            styled(child: [Tensor<B, 2>], ..),
                             [ ],
                             ),
                           ),
                          [ ],
                        sequence(
                      h(amount: 7pt),
                             [ ],
```

```
styled(child: equation(block: false, body: [→]), ..),
                        [ ],
                    h(amount: 7pt),
                        ),
                       [ ],
                       ),
                     sequence(
                       [ ],
                       box(
                  fill: rgb("#dceefb"),
                stroke: 0.8pt + rgb("#20639b"),
                  radius: 0% + 4pt,
                  inset: 0% + 7pt,
                  body: sequence(
                         [ ],
                        align(
                  alignment: center,
              body: strong(body: [Neural network]),
                          ),
                    linebreak(),
                       [ ],
               styled(child: [prediction], ..),
                       [ ],
                       ),
                      ),
                     [ ],
                     [],
                     [ ],
                     ),
                  )).join()
```

Burn 0.20.1 · Rust · NdArray · Polars

How to use this tutorial

This is a practical introduction to Burn 0.20.1. Work through the sections in order and type the examples yourself. The tutorial starts with tensors, then uses those tensors to build and train a small neural network that predicts a taxi fare from trip distance.

The examples use the same style as `learning_burn/taxis.rs`: explicit shapes, generic backends, `Result`-based errors, and tests for data-processing code. The diagrams are generated inside this Typst document, so the document remains useful even when viewed away from the repository.

Run this project

From the repository root:

```
cargo fmt
cargo check --bin taxis
cargo test --test taxi_learning
cargo run --bin taxis
```

The taxi program reads `learning_burn/yellow_tripdata_2024-01.parquet`, selects `trip_distance` and `fare_amount`, filters invalid rows, trains on a short 20,000-row sample, evaluates on validation data, prints predictions, and saves a model to `learning_burn/taxi_model.bin`.

The model file is generated output. It is ignored by Git and can be deleted and regenerated at any time.

> **Learning path**
> First learn how a tensor is created and reshaped. Then learn how operations preserve or change shapes. Finally connect those operations to the model and training loop.

1. Your first Burn program

Create a small Rust file with these imports:

```
use burn::{
    backend::NdArray,
    tensor::{Tensor, TensorData},
};

type B = NdArray;

fn main() {
    let device = Default::default();
    let tensor = Tensor::<B, 2>::from_data([[1.0, 2.0], [3.0, 4.0]], &device);
    println!("{tensor}");
}
```

Run it as a Cargo binary. A Burn tensor is written as:

```
Tensor<Backend, Rank, Kind>
```

- `Backend` says where computation runs. `NdArray` is a portable CPU backend.
- `Rank` is the number of dimensions. A matrix is rank 2; a batch of images might be rank 4.
- `Kind` is usually inferred as floating point, integer, or boolean.

The shape sizes are checked at runtime. The rank is part of the Rust type, so a rank-2 tensor cannot accidentally be passed where a rank-3 tensor is required.

```
(
sequence(
```

```
                    [ ],
                    box(
                fill: rgb("#dceefb"),
            stroke: 0.8pt + rgb("#20639b"),
                   radius: 0% + 4pt,
                   inset: 0% + 7pt,
                    body: sequence(
                           [ ],
align(alignment: center, body: strong(body: [Backend])),
                      linebreak(),
                           [ ],
          styled(child: [NdArray], ..),
                           [ ],
                           ),
                         ),
                        [ ],
                      sequence(
                    h(amount: 7pt),
                           [ ],
 styled(child: equation(block: false, body: [→]), ..),
                           [ ],
                    h(amount: 7pt),
                           ),
                        [ ],
                        ),
                     sequence(
                        [ ],
                        box(
                fill: rgb("#dceefb"),
            stroke: 0.8pt + rgb("#20639b"),
                   radius: 0% + 4pt,
                   inset: 0% + 7pt,
                    body: sequence(
                           [ ],
  align(alignment: center, body: strong(body: [Rank])),
                      linebreak(),
                           [ ],
       styled(child: [2 dimensions], ..),
                           [ ],
                           ),
                         ),
                        [ ],
                      sequence(
                    h(amount: 7pt),
                           [ ],
 styled(child: equation(block: false, body: [→]), ..),
                           [ ],
                    h(amount: 7pt),
                           ),
                        [ ],
                        ),
                     sequence(
                        [ ],
                        box(
                fill: rgb("#dceefb"),
            stroke: 0.8pt + rgb("#20639b"),
                   radius: 0% + 4pt,
```

```
                    inset: 0% + 7pt,
                    body: sequence(
                        [ ],
                align(alignment: center, body: strong(body: [Kind])),
                    linebreak(),
                        [ ],
                styled(child: [Float], ..),
                        [ ],
                        ),
                    ),
                    [ ],
                  sequence(
                h(amount: 7pt),
                        [ ],
            styled(child: equation(block: false, body: [→]), ..),
                        [ ],
                h(amount: 7pt),
                    ),
                    [ ],
                    ),
                  sequence(
                        [ ],
                        box(
                    fill: rgb("#dceefb"),
                stroke: 0.8pt + rgb("#20639b"),
                    radius: 0% + 4pt,
                    inset: 0% + 7pt,
                    body: sequence(
                        [ ],
                align(alignment: center, body: strong(body: [Shape])),
                    linebreak(),
                        [ ],
                styled(child: [[2, 2]], ..),
                        [ ],
                        ),
                    ),
                    [ ],
                    [],
                    [ ],
                    ),
                )).join()
```

For the taxi model, the training backend is:

```
use burn::backend::{Autodiff, NdArray};

type TrainBackend = Autodiff<NdArray>;
```

`Autodiff<NdArray>` wraps the CPU backend with the graph needed for `loss.backward()`.

> **Backend mental model**
> The model describes **what** computation occurs. The backend decides **where** it occurs. This separation lets the same generic module be tested on CPU and later moved to another backend.

2. Tensor initialization

All examples below assume:

```
use burn::{
    backend::NdArray,
    tensor::{Distribution, Int, Tensor, TensorData, TensorCreationOptions},
};

type B = NdArray;
let device = Default::default();
```

`from_data`: explicit values

Use `from_data` when values are already in a Rust array, vector, or `TensorData`.

```
let matrix = Tensor::<B, 2>::from_data(
    [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]],
    &device,
);

let values = vec![1.0_f32, 2.0, 3.0, 4.0];
let vector = Tensor::<B, 2>::from_data(
    TensorData::new(values, [2, 2]),
    &device,
);
```

Use `TensorData::new` when data comes from a file or iterator and you know the shape separately.

from_data: shape [2, 3]

1	2	3
4	5	6

`from_floats`: readable floating-point data

```
let vector = Tensor::<B, 1>::from_floats([1.0, 2.0, 3.0], &device);
let matrix = Tensor::<B, 2>::from_floats([[1.0, 2.0], [3.0, 4.0]], &device);
```

For most ML features, labels, weights, and activations, floating-point tensors are the right starting point.

`from_ints`: integer tensors

Use integer tensors for token IDs, class IDs, indexes, and other discrete values:

```
let labels = Tensor::<B, 2, Int>::from_ints([[0, 1], [2, 1]], &device);
let as_float = labels.float();
```

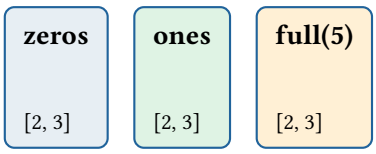
Integer tensors are not interchangeable with floating-point tensors. Convert deliberately when an operation requires another kind.

Filled tensors

```
let zeros = Tensor::<B, 2>::zeros([2, 3], &device);
let ones = Tensor::<B, 2>::ones([2, 3], &device);
let fives = Tensor::<B, 2>::full([2, 3], 5.0, &device);

let zeros_like = matrix.zeros_like();
let ones_like = matrix.ones_like();
let full_like = matrix.full_like(-1.0);
```

These are useful for initialization, padding, masks, accumulators, and test fixtures.



`empty`

```
let uninitialized = Tensor::<B, 2>::empty([2, 3], &device);
```

`empty` allocates storage without filling it with a meaningful value. Prefer `zeros`, `ones`, or `full` unless you have a specific performance-sensitive reason to use it. Never read an empty tensor expecting initialized values.

Random tensors

```
let uniform = Tensor::<B, 2>::random(
    [2, 3],
    Distribution::Uniform(0.0, 1.0),
    &device,
);
let normal = Tensor::<B, 2>::random(
    [2, 3],
    Distribution::Normal(0.0, 1.0),
    &device,
);
let random_with_existing_shape =
    matrix.random_like(Distribution::Normal(0.0, 1.0));
```

Random initialization is common for experiments and model parameters. Reproducibility requires controlling the random seed or using deterministic initialization where appropriate. Burn's neural-network layers provide their own sensible initializers, so do not manually replace them unless you understand why.

`arange`, `arange_step`, and `eye`

```
let positions = Tensor::<B, 1, Int>::arange(0..5, &device);
let every_second = Tensor::<B, 1, Int>::arange_step(0..10, 2, &device);
let identity = Tensor::<B, 2>::eye(3, &device);
```

`arange` is useful for positions, indexes, and batch construction. `eye` creates an identity matrix for linear algebra, residual connections, and tests.

eye(3)

1	0	0
0	1	0
0	0	1

Creation options

Most examples use `&device` because it is concise. Burn also accepts `TensorCreationOptions` when explicit device or dtype control is useful:

```
let options = TensorCreationOptions::<B>::new(device.clone());
let tensor = Tensor::<B, 2>::ones([2, 2], options);
```

3. Inspecting tensors

Burn's `Display` implementation intentionally prints values plus shape, device, backend, kind, and dtype:

```
println!("{tensor}");
```

To read only the values:

```
let values = tensor
    .into_data()
    .to_vec::<f32>()
    .expect("tensor data should be readable");
println!("{values:?}");
```

For a matrix, preserve rows yourself:

```
let values = matrix.into_data().to_vec::<f32>().unwrap();
for row in values.chunks(3) {
    println!("{row:?}");
}
```

Useful inspection methods:

```
println!("shape: {:?}", matrix.dims());
println!("rank: {}", matrix.dims().len());
println!("device: {:?}", matrix.device());
```

`into_data()` or `to_data()` may synchronize an accelerator with the CPU. Use them for logging, metrics, and final output, not on every operation in a hot training loop.

4. Shape and layout operations

Shape mistakes are among the most common ML bugs. Check shapes early:

```
let batch = Tensor::<B, 2>::ones([4, 3], &device);
assert_eq!(batch.dims(), [4, 3]);
```

Operation	Input	Output	Why it matters
reshape([2, 2, 3])	[2, 6]	[2, 2, 3]	Change layout; preserve element count
flatten(1, 3)	[32, 1, 28, 28]	[32, 784]	Prepare image features for Linear
unsqueeze()	[3]	[1, 3]	Create a one-item batch
transpose()	[2, 3]	[3, 2]	Swap matrix axes
expand([4, 3])	[1, 3]	[4, 3]	Broadcast a batch-wise value

`reshape`

```
let tensor = Tensor::<B, 2>::ones([2, 6], &device);
let reshaped: Tensor<B, 3> = tensor.reshape([2, 2, 3]);
let inferred: Tensor<B, 2> = reshaped.reshape([2, -1]);
```

The `-1` dimension is inferred. Burn rejects a shape whose element count does not match.

`flatten`

```
let images = Tensor::<B, 4>::ones([32, 1, 28, 28], &device);
let features: Tensor<B, 2> = images.flatten(1, 3);
assert_eq!(features.dims(), [32, 784]);
```

`unsqueeze` and `squeeze`

```
let vector = Tensor::<B, 1>::ones([3], &device);
let column: Tensor<B, 2> = vector.unsqueeze();
let restored: Tensor<B, 1> = column.squeeze();
```

`transpose`, `t`, and `swap_dims`

```
let matrix = Tensor::<B, 2>::from_floats(
    [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]],
    &device,
);
let transposed = matrix.clone().transpose(); // [3, 2]
let same = matrix.clone().t();               // alias
let swapped = matrix.swap_dims(0, 1);        // [3, 2]
```

For tensors with more than two dimensions, `transpose` swaps the last two dimensions. Use `swap_dims` when naming dimensions explicitly.

`expand`

```
let bias = Tensor::<B, 2>::ones([1, 3], &device);
let batch_bias: Tensor<B, 2> = bias.expand([4, 3]);
```

Use broadcasting for batch-wise constants, but verify that the expansion matches the intended semantics.

5. Arithmetic and mathematical operations

Burn overloads common operators:

```
let a = Tensor::<B, 2>::from_floats([[1.0, 2.0], [3.0, 4.0]], &device);
let b = Tensor::<B, 2>::ones([2, 2], &device);

let add = a.clone() + b.clone();
let subtract = a.clone() - b.clone();
let multiply = a.clone() * b.clone();
let divide = a / b;
```

Scalar methods are useful when one side is a Rust number:

```
let shifted = a.clone().add_scalar(1.0);
let scaled = a.clone().mul_scalar(2.0);
let divided = a.clone().div_scalar(2.0);
```

Common mathematical methods:

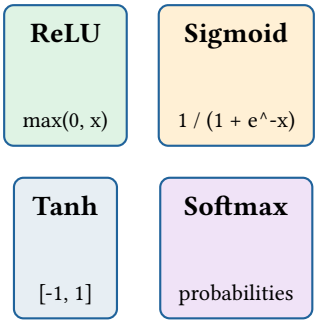
```
let absolute = a.clone().abs();
let bounded = a.clone().clamp(0.0, 1.0);
let lower_bounded = a.clone().clamp_min(0.0);
let upper_bounded = a.clone().clamp_max(1.0);
let squared = a.clone().powi_scalar(2);
let square_root = a.clone().sqrt();
```

```rust
let reciprocal = a.clone().recip();
let exponential = a.clone().exp();
let logarithm = a.log();
```

Activation functions

```rust
use burn::tensor::activation;

let relu = activation::relu(a.clone());
let sigmoid = activation::sigmoid(a.clone());
let tanh = activation::tanh(a.clone());
let probabilities = activation::softmax(a, 1);
let log_probabilities = activation::log_softmax(probabilities, 1);
```



Matrix multiplication

```rust
let left = Tensor::<B, 2>::ones([4, 3], &device);
let right = Tensor::<B, 2>::ones([3, 2], &device);
let output = left.matmul(right);
assert_eq!(output.dims(), [4, 2]);
```

The inner dimensions must agree:

```
(
sequence(
[ ],
box(
fill: rgb("#dceefb"),
stroke: 0.8pt + rgb("#20639b"),
radius: 0% + 4pt,
inset: 0% + 7pt,
body: sequence(
[ ],
align(alignment: center, body: strong(body: [left])),
linebreak(),
[ ],
styled(child: [[batch, features]], ..),
[ ],
),
),
[ ],
sequence(
h(amount: 7pt),
[ ],
styled(child: equation(block: false, body: [→]), ..),
[ ],
h(amount: 7pt),
```

```
                        ),
                      [ ],
                      ),
                    sequence(
                        [ ],
                      box(
                 fill: rgb("#fff0d5"),
           stroke: 0.8pt + rgb("#20639b"),
                  radius: 0% + 4pt,
                  inset: 0% + 7pt,
                   body: sequence(
                         [ ],
align(alignment: center, body: strong(body: [matmul])),
                     linebreak(),
                         [ ],
         styled(child: [features agree], ..),
                         [ ],
                         ),
                      ),
                      [ ],
                    sequence(
                  h(amount: 7pt),
                         [ ],
styled(child: equation(block: false, body: [→]), ..),
                         [ ],
                  h(amount: 7pt),
                        ),
                      [ ],
                      ),
                    sequence(
                        [ ],
                      box(
                 fill: rgb("#dceefb"),
           stroke: 0.8pt + rgb("#20639b"),
                  radius: 0% + 4pt,
                  inset: 0% + 7pt,
                   body: sequence(
                         [ ],
 align(alignment: center, body: strong(body: [right])),
                     linebreak(),
                         [ ],
       styled(child: [[features, outputs]], ..),
                         [ ],
                         ),
                      ),
                      [ ],
                    sequence(
                  h(amount: 7pt),
                         [ ],
styled(child: equation(block: false, body: [→]), ..),
                         [ ],
                  h(amount: 7pt),
                        ),
                      [ ],
                      ),
                    sequence(
                        [ ],
```

```
                box(
              fill: rgb("#dff3e4"),
            stroke: 0.8pt + rgb("#20639b"),
              radius: 0% + 4pt,
              inset: 0% + 7pt,
              body: sequence(
                    [ ],
align(alignment: center, body: strong(body: [output])),
                linebreak(),
                    [ ],
        styled(child: [[batch, outputs]], ..),
                    [ ],
                    ),
                  ),
                 [ ],
                 [],
                [ ],
                ),
             )).join()
```

6. Reductions and statistics

Reductions turn many values into summaries:

```
let values = Tensor::<B, 2>::from_floats(
    [[1.0, 2.0], [3.0, 4.0]],
    &device,
);

let total = values.clone().sum();
let average = values.clone().mean();
let minimum = values.clone().min();
let maximum = values.clone().max();
```

Reduce along a dimension when preserving the rest of the shape:

```
let column_means = values.clone().mean_dim(0); // [1, 2]
let row_sums = values.clone().sum_dim(1);      // [2, 1]
let column_maximums = values.clone().max_dim(0);
let row_minimums = values.min_dim(1);
```

Other useful reductions include `argmax`, `min_abs`, `max_abs`, and variance/statistics helpers. Check the resulting shape after each reduction; whether a dimension is retained often matters to the next layer.

7. Indexing, concatenating, and masking

Slices

```
let batch = Tensor::<B, 2>::from_floats(
    [[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]],
    &device,
);

let first_two_rows = batch.clone().slice([0..2, 0..2]);
let last_column = batch.clone().slice([0..3, 1..2]);
```

Select rows by indexes

```
let indexes = Tensor::<B, 1, Int>::from_ints([2, 0], &device);
let selected = batch.select(0, indexes);
```

Concatenate, stack, and chunk

```
let first = Tensor::<B, 2>::ones([2, 3], &device);
let second = Tensor::<B, 2>::zeros([2, 3], &device);
let joined = Tensor::cat(vec![first, second], 0); // [4, 3]

let a = Tensor::<B, 2>::ones([2, 3], &device);
let b = Tensor::<B, 2>::zeros([2, 3], &device);
let stacked: Tensor<B, 3> = Tensor::stack(vec![a, b], 0); // [2, 2, 3]
let parts = joined.chunk(2, 0);
```

Operation	Input	Output	Why it matters
cat(..., 0)	[2, 3] + [2, 3]	[4, 3]	Join on an existing axis
stack(..., 0)	[2, 3] + [2, 3]	[2, 2, 3]	Create a new axis
chunk(2, 0)	[4, 3]	[2, 3] each	Split a batch

Triangular masks and `mask_fill`

```
use burn::tensor::Bool;

let mask = Tensor::<B, 2, Bool>::tril_mask([3, 3], 0, &device);
let scores = Tensor::<B, 2>::ones([3, 3], &device);
let masked = scores.mask_fill(mask, f32::NEG_INFINITY);
```

tril mask: visible causal positions

1	0	0
1	1	0
1	1	1

For causal attention, `tril_mask` keeps the current and previous positions visible. `triu_mask` creates the opposite triangular mask. `diag_mask` creates a diagonal mask. `tril` and `triu` directly replace values outside the selected triangle with zero.

8. Ownership and `clone()`

Tensor operations usually consume `self`:

```
let scaled = tensor.mul_scalar(2.0);
// tensor is moved and cannot be used here
```

When the same tensor is needed twice, clone it:

```
let left = tensor.clone() + other;
let right = tensor.clone().transpose();
```

This is a Rust ownership rule, not a Burn-specific requirement. In a training loop, avoid cloning more than necessary, but do not fight the type system. More importantly, avoid repeated `into_data()` calls: reading data back can be much more expensive than a device-side tensor operation.

A useful rule
Clone when you need another computation branch. Do not clone merely to print a tensor; use `into_data()` at the point where values genuinely need to return to Rust.

9. Automatic differentiation

Use an autodiff backend when you need gradients:

```rust
use burn::{
    backend::{Autodiff, NdArray},
    tensor::Tensor,
};

type TrainBackend = Autodiff<NdArray>;
let device = Default::default();
let input = Tensor::<TrainBackend, 2>::from_floats([[2.0], [3.0]], &device);
let weight = Tensor::<TrainBackend, 2>::ones([1, 1], &device).require_grad();
let output = input.matmul(weight.clone());
let loss = output.clone().mean();
let gradients = loss.backward();
let weight_gradient = weight.grad(&gradients);
```

Visualized:

```
                                    (
                                  sequence(
                                      [ ],
                                      box(
                                 fill: rgb("#dceefb"),
                               stroke: 0.8pt + rgb("#20639b"),
                                      radius: 0% + 4pt,
                                      inset: 0% + 7pt,
                                       body: sequence(
                                             [ ],
align(alignment: center, body: strong(body: [input])),
                                        linebreak(),
                                             [ ],
                             styled(child: [[2, 1]], ..),
                                             [ ],
                                             ),
                                           ),
                                          [ ],
                                        sequence(
                                      h(amount: 7pt),
                                             [ ],
styled(child: equation(block: false, body: [→]), ..),
                                             [ ],
                                      h(amount: 7pt),
                                           ),
                                          [ ],
                                          ),
                                        sequence(
                                          [ ],
                                          box(
                                 fill: rgb("#fff0d5"),
                               stroke: 0.8pt + rgb("#20639b"),
                                      radius: 0% + 4pt,
```

```
                    inset: 0% + 7pt,
                    body: sequence(
                        [ ],
align(alignment: center, body: strong(body: [weight])),
                      linebreak(),
                        [ ],
                styled(child: [[1, 1]], ..),
                        [ ],
                        ),
                      ),
                     [ ],
                    sequence(
                    h(amount: 7pt),
                        [ ],
styled(child: equation(block: false, body: [→]), ..),
                        [ ],
                    h(amount: 7pt),
                        ),
                      [ ],
                      ),
                     sequence(
                        [ ],
                        box(
                  fill: rgb("#dceefb"),
             stroke: 0.8pt + rgb("#20639b"),
                    radius: 0% + 4pt,
                    inset: 0% + 7pt,
                    body: sequence(
                        [ ],
align(alignment: center, body: strong(body: [forward])),
                      linebreak(),
                        [ ],
                styled(child: [matmul], ..),
                        [ ],
                        ),
                      ),
                     [ ],
                    sequence(
                    h(amount: 7pt),
                        [ ],
styled(child: equation(block: false, body: [→]), ..),
                        [ ],
                    h(amount: 7pt),
                        ),
                      [ ],
                      ),
                     sequence(
                        [ ],
                        box(
                  fill: rgb("#f0e3f7"),
             stroke: 0.8pt + rgb("#20639b"),
                    radius: 0% + 4pt,
                    inset: 0% + 7pt,
                    body: sequence(
                        [ ],
 align(alignment: center, body: strong(body: [loss])),
                      linebreak(),
```

```
                [ ],
            styled(child: [mean], ..),
                [ ],
                ),
                ),
                [ ],
                sequence(
                h(amount: 7pt),
                [ ],
        styled(child: equation(block: false, body: [→]), ..),
                [ ],
                h(amount: 7pt),
                ),
                [ ],
                ),
                sequence(
                [ ],
                box(
            fill: rgb("#dff3e4"),
        stroke: 0.8pt + rgb("#20639b"),
            radius: 0% + 4pt,
            inset: 0% + 7pt,
            body: sequence(
                [ ],
align(alignment: center, body: strong(body: [backward])),
                linebreak(),
                [ ],
            styled(child: [gradient], ..),
                [ ],
                ),
                ),
                [ ],
                [],
                [ ],
                ),
            )).join()
```

In practice, Burn modules and optimizers manage parameter gradients:

```
let prediction = model.forward(train_x.clone());
let loss = loss_fn.forward(prediction, train_y.clone(), Reduction::Mean);
let gradients = GradientsParams::from_grads(loss.backward(), &model);
model = optimizer.step(learning_rate, model, gradients);
```

That is the core of the `train` function in `taxis.rs`.

10. Building a Burn neural network

`TaxiModel` is intentionally small:

```
                (
                sequence(
                [ ],
                box(
            fill: rgb("#dceefb"),
        stroke: 0.8pt + rgb("#20639b"),
            radius: 0% + 4pt,
            inset: 0% + 7pt,
```

```
                body: sequence(
                    [ ],
                  align(
                alignment: center,
            body: strong(body: [Trip distance]),
                    ),
                linebreak(),
                    [ ],
             styled(child: [[batch, 1]], ..),
                    [ ],
                    ),
                  ),
                 [ ],
                sequence(
                h(amount: 7pt),
                    [ ],
styled(child: equation(block: false, body: [→]), ..),
                    [ ],
                h(amount: 7pt),
                    ),
                 [ ],
                 ),
                sequence(
                    [ ],
                    box(
              fill: rgb("#fff0d5"),
            stroke: 0.8pt + rgb("#20639b"),
                radius: 0% + 4pt,
                inset: 0% + 7pt,
                body: sequence(
                    [ ],
align(alignment: center, body: strong(body: [Linear])),
                linebreak(),
                    [ ],
           styled(child: [1 → 8], ..),
                    [ ],
                    ),
                  ),
                 [ ],
                sequence(
                h(amount: 7pt),
                    [ ],
styled(child: equation(block: false, body: [→]), ..),
                    [ ],
                h(amount: 7pt),
                    ),
                 [ ],
                 ),
                sequence(
                    [ ],
                    box(
              fill: rgb("#dff3e4"),
            stroke: 0.8pt + rgb("#20639b"),
                radius: 0% + 4pt,
                inset: 0% + 7pt,
                body: sequence(
                    [ ],
```

```
align(alignment: center, body: strong(body: [ReLU])),
                    linebreak(),
                        [ ],
            styled(child: [activation], ..),
                        [ ],
                        ),
                      ),
                     [ ],
                   sequence(
                  h(amount: 7pt),
                        [ ],
styled(child: equation(block: false, body: [→]), ..),
                        [ ],
                  h(amount: 7pt),
                        ),
                     [ ],
                     ),
                   sequence(
                        [ ],
                     box(
                fill: rgb("#fff0d5"),
            stroke: 0.8pt + rgb("#20639b"),
                   radius: 0% + 4pt,
                   inset: 0% + 7pt,
                    body: sequence(
                           [ ],
align(alignment: center, body: strong(body: [Linear])),
                    linebreak(),
                        [ ],
               styled(child: [8 → 1], ..),
                        [ ],
                        ),
                      ),
                     [ ],
                   sequence(
                  h(amount: 7pt),
                        [ ],
styled(child: equation(block: false, body: [→]), ..),
                        [ ],
                  h(amount: 7pt),
                        ),
                     [ ],
                     ),
                   sequence(
                        [ ],
                     box(
                fill: rgb("#f0e3f7"),
            stroke: 0.8pt + rgb("#20639b"),
                   radius: 0% + 4pt,
                   inset: 0% + 7pt,
                    body: sequence(
                           [ ],
                         align(
                    alignment: center,
          body: strong(body: [Predicted fare]),
                         ),
                   linebreak(),
```

```
                    [ ],
styled(child: [[batch, 1]], ..),
                    [ ],
                    ),
                ),
            [ ],
            [],
            [ ],
            ),
    )).join()
```

The Rust structure is:

```rust
#[derive(Module, Debug)]
struct TaxiModel<B: Backend> {
    hidden: Linear<B>,
    output: Linear<B>,
}

impl<B: Backend> TaxiModel<B> {
    fn new(device: &B::Device) -> Self {
        Self {
            hidden: LinearConfig::new(1, 8).init(device),
            output: LinearConfig::new(8, 1).init(device),
        }
    }

    fn forward(&self, input: Tensor<B, 2>) -> Tensor<B, 2> {
        self.output
            .forward(activation::relu(self.hidden.forward(input)))
    }
}
```

The important design rules are:

1. `new` creates parameters on a device.
2. `forward` consumes an input tensor and returns the prediction tensor.
3. Input and output shapes are documented and tested.
4. The model is generic over `B`, so it is not permanently tied to one backend.

11. The taxi regression project

Step 1: Load only the columns you need

Polars reads the Parquet file:

```rust
let frame = LazyFrame::scan_parquet(path, ScanArgsParquet::default())?
    .select([col("trip_distance"), col("fare_amount")])
    .collect()?;
```

Do not load every column by default. Selecting required columns reduces memory use and makes the feature contract clear.

Step 2: Validate rows before creating tensors

The example keeps only finite, positive distances and fares:

```rust
if distance.is_finite()
    && fare.is_finite()
    && distance > 0.0
```

```
    && fare > 0.0
{
    distances.push(distance);
    fares.push(fare);
}
```

Invalid data should be handled at the boundary, not discovered as a mysterious training failure.

Step 3: Split before fitting normalization

The example splits into training and validation data, then computes mean and standard deviation from training data only:

$$x_{\text{normalized}} = \frac{x - \mu}{\sigma}$$

Using validation values to fit preprocessing is data leakage. The same training normalizers must be reused at validation and inference time.

```
                                    (
                                  sequence(
                                      [ ],
                                      box(
                                 fill: rgb("#dceefb"),
                               stroke: 0.8pt + rgb("#20639b"),
                                 radius: 0% + 4pt,
                                  inset: 0% + 7pt,
                                  body: sequence(
                                        [ ],
align(alignment: center, body: strong(body: [All rows])),
                                    linebreak(),
                                      [ ],
                            styled(child: [[N, 2]], ..),
                                      [ ],
                                      ),
                                    ),
                                    [ ],
                                  sequence(
                                 h(amount: 7pt),
                                      [ ],
 styled(child: equation(block: false, body: [→]), ..),
                                      [ ],
                                 h(amount: 7pt),
                                      ),
                                    [ ],
                                    ),
                                  sequence(
                                      [ ],
                                      box(
                                 fill: rgb("#fff0d5"),
                               stroke: 0.8pt + rgb("#20639b"),
                                 radius: 0% + 4pt,
                                  inset: 0% + 7pt,
                                  body: sequence(
                                        [ ],
 align(alignment: center, body: strong(body: [Filter])),
                                    linebreak(),
                                      [ ],
                         styled(child: [finite, positive], ..),
```

```
                                [ ],
                                ),
                                ),
                               [ ],
                             sequence(
                           h(amount: 7pt),
                                [ ],
  styled(child: equation(block: false, body: [→]), ..),
                                [ ],
                           h(amount: 7pt),
                                ),
                               [ ],
                               ),
                             sequence(
                                [ ],
                               box(
                         fill: rgb("#e7eef3"),
                    stroke: 0.8pt + rgb("#20639b"),
                           radius: 0% + 4pt,
                           inset: 0% + 7pt,
                           body: sequence(
                                  [ ],
   align(alignment: center, body: strong(body: [Split])),
                              linebreak(),
                                [ ],
                  styled(child: [80% / 20%], ..),
                                [ ],
                                ),
                                ),
                               [ ],
                             sequence(
                           h(amount: 7pt),
                                [ ],
  styled(child: equation(block: false, body: [→]), ..),
                                [ ],
                           h(amount: 7pt),
                                ),
                               [ ],
                               ),
                             sequence(
                                [ ],
                               box(
                         fill: rgb("#dff3e4"),
                    stroke: 0.8pt + rgb("#20639b"),
                           radius: 0% + 4pt,
                           inset: 0% + 7pt,
                           body: sequence(
                                  [ ],
 align(alignment: center, body: strong(body: [Normalize])),
                              linebreak(),
                                [ ],
                  styled(child: [train μ, σ], ..),
                                [ ],
                                ),
                                ),
                               [ ],
                             sequence(
```

```
                    h(amount: 7pt),
                        [ ],
styled(child: equation(block: false, body: [→]), ..),
                        [ ],
                    h(amount: 7pt),
                        ),
                       [ ],
                       ),
                     sequence(
                       [ ],
                       box(
                fill: rgb("#f0e3f7"),
              stroke: 0.8pt + rgb("#20639b"),
                radius: 0% + 4pt,
                inset: 0% + 7pt,
                body: sequence(
                    [ ],
align(alignment: center, body: strong(body: [Train])),
                  linebreak(),
                       [ ],
          styled(child: [Burn model], ..),
                       [ ],
                       ),
                     ),
                    [ ],
                    [],
                    [ ],
                    ),
                )).join()
```

Step 4: Train and measure in understandable units

The model trains against normalized targets. The validation loss is converted back to dollar RMSE:

```
let validation_rmse = validation_loss.sqrt() * fare_normalizer.std;
```

Metrics should be understandable to the person using the model. `$7.11 RMSE` is more useful than `0.42 normalized MSE`.

Step 5: Save and reload

```
let recorder = BinFileRecorder::<FullPrecisionSettings>::default();
model.save_file("learning_burn/taxi_model", &recorder)?;

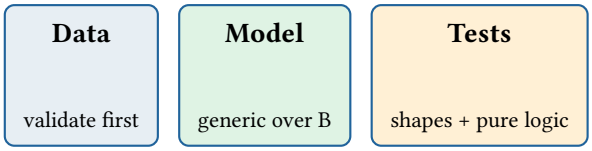
let loaded = TaxiModel::new(&device).load_file(
    "learning_burn/taxi_model",
    &recorder,
    &device,
)?;
```

Saving only weights is not enough for a production model. Save feature names, normalization statistics, hyper-parameters, Burn version, and model version alongside the weights.

> **Why this model is intentionally simple**
> One feature makes the tensor shapes and regression loop visible. A production fare model should add location, time, passenger count, tolls, and other fields only after this baseline is understood.

12. Rust ML-engineering practices

- Keep data loading, validation, preprocessing, model code, training, and inference in separate functions.
- Return `Result` from reusable functions and add context to errors at the executable boundary.
- Use traits and generics to keep model code backend-independent.
- Test pure functions such as filtering, splitting, and normalization without loading the large dataset.
- Add shape tests for every model boundary.
- Keep training deterministic while learning; add seeded shuffling before comparing experiments.
- Do not read tensors back to CPU inside every training step.
- Run `cargo fmt`, `cargo check`, `cargo test`, and `cargo clippy`.
- Treat generated weights, logs, and plots as artifacts rather than source files.



13. Exercises

1. Replace the row-order split with a deterministic seeded shuffle.
2. Add a test that verifies the model output shape is `[batch, 1]`.
3. Add `passenger_count` as a second input feature and change `LinearConfig::new(1, 8)` to `LinearConfig::new(2, 8)`.
4. Add a held-out test split and report train, validation, and test RMSE.
5. Save and reload the normalizers together with the model.
6. Add mini-batches instead of using all training rows in one batch.
7. Add a command-line argument for the dataset path and number of epochs.
8. Compare a one-layer linear model with the two-layer model.
9. Try the same model with WGPU after the NdArray version is understood.

API quick reference

Operation	Input	Output	Why it matters
`from_data`	Rust values	chosen rank	Explicit tensor contents
`from_floats`	float arrays	chosen rank	Readable numeric examples
`from_ints`	integer arrays	chosen rank	IDs and indexes
`zeros` / `ones`	shape	same rank	Constant initialization
`full`	shape + value	same rank	Sentinels and fixtures
`empty`	shape	same rank	Uninitialized storage; use carefully
`random`	shape + distribution	same rank	Experiments and initialization
`arange`	range	rank 1	Positions and indexes
`eye`	size	rank 2	Identity matrix
`reshape`	new shape	new rank	Change layout
`flatten`	dimension range	new rank	Flatten feature dimensions
`unsqueeze`	existing tensor	+1 rank	Add a batch/channel axis
`squeeze`	existing tensor	-1 or more	Remove size-one axes
`transpose`	existing tensor	same rank	Swap final two axes
`matmul`	two tensors	same output rank	Linear algebra
`mean` / `sum`	tensor	rank 1	Global reduction
`mean_dim` / `sum_dim`	tensor + axis	same rank	Axis reduction
`slice` / `select`	tensor + indexes	same rank	Read a subset
`cat` / `stack`	tensor list + axis	same/new rank	Join tensors
`mask_fill`	tensor + Bool mask	same rank	Conditional replacement
`tril` / `triu`	tensor + diagonal	same rank	Triangular structure

The fastest way to learn Burn is to change one shape, operation, or layer at a time, run the tests, and inspect the result.